VIDYA — Master Systems Architecture Handbook

Complete Educational Guide: RAG Pipeline, LangGraph State Engine, Safe AST Evaluator & Web Search

---

VIDYA — Master Systems Architecture & Engineering Handbook

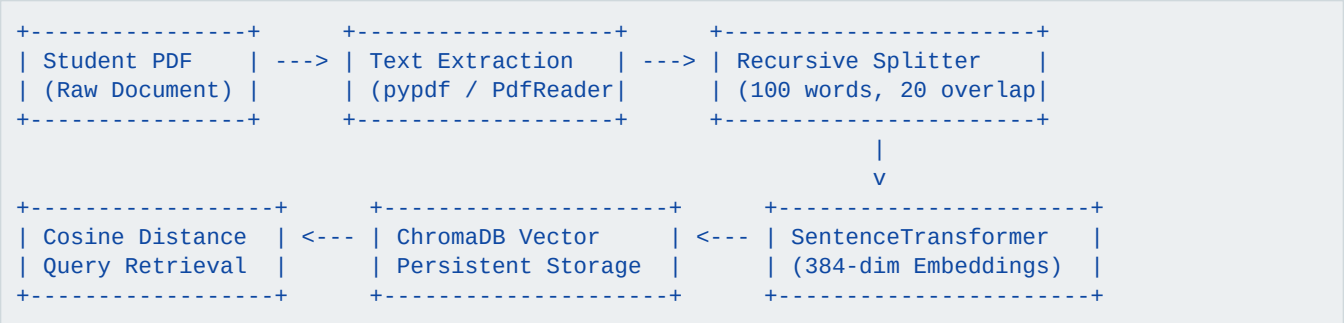
An In-Depth Engineering Guide covering RAG, LangGraph State Engines, Safe AST Tools, and Web Search Integration

---

Chapter 1: V1 — RAG Core Architecture & Vector Retrieval

1. High-Level Architecture Overview

VIDYA V1 is built on the **Retrieval-Augmented Generation (RAG)** architecture. RAG bridges the gap between static LLM knowledge and specific custom documents (like a student's uploaded PDF textbook).



---

2. Core Concepts & Topics Used

A. PDF Loaders (`pypdf`)

- **What it does**: Reads raw binary PDF files and extracts raw text string page-by-page.

- **Why we used** `pypdf`: Lightweight, zero native C-dependency overhead, pure Python, fast string extraction.

- **Alternatives**:

- `pdfplumber` / `PyMuPDF (fitz)`: Slower, heavier C-bindings. Useful for tables/images, but overkill for pure text textbooks.

- Raw string reads (`open("file.pdf")`): Impossible because PDFs store text encoded inside stream objects and compressed binary structures.

B. Text Chunking (`split_text`)

- **What it does**: Splits a long document into smaller overlapping text segments (e.g., 100 words with 20 words overlap).

- **Why Chunking is Required**:

1. **LLM Context Limit & Noise**: Sending an entire 500-page textbook to an LLM dilutes attention and costs high token fees.

2. **Semantic Granularity**: Vector search works best on concise, focused passages (ideas), not entire books.

- **Why Overlap is Crucial**: If a sentence like *"Supervised learning requires labeled data"* gets cut exactly between *"Supervised learning"* (Chunk 1) and *"requires labeled data"* (Chunk 2), the context is broken. Overlap ensures adjacent chunks share edge context.

- **Alternatives**:

- **No Chunking**: Fails due to LLM context overflow or loss of accuracy (needle in a haystack problem).

- **Character Splitting**: Cuts words in half (e.g., "Supervi" and "sed"). Word-based or token-based sliding windows are far superior.

C. Dense Vector Embeddings (`sentence-transformers` - `all-MiniLM-L6-v2`)

- **What it does**: Converts text chunks into 384-dimensional floating-point vectors representing semantic meaning.

- **Why** `all-MiniLM-L6-v2`:

- Runs locally on CPU in milliseconds (no API cost).

- High accuracy for semantic similarity tasks.

- **Why Embeddings vs Keyword Search (TF-IDF / BM25)**:

- Keyword search fails when different words share the same meaning (e.g., "automobile" vs "car", "classification" vs "categorization").

- Vector embeddings place semantically similar concepts close together in 384-dimensional vector space regardless of exact word matches.

D. Vector Database (`ChromaDB`)

- **What it does**: Stores vector embeddings along with text chunks on disk (`./chroma_db`) and builds an HNSW (Hierarchical Navigable Small World) index for ultra-fast similarity searches.

- **Why ChromaDB**:

- Embedded, persistent on local disk, zero external server setup required.

- Native Python integration.

- **Alternatives**:

- **In-Memory Python List**: Simple, but scanning `O(N)` vectors manually on every user question is extremely slow and lost when the process exits.

- **Pinecone / Weaviate**: Require cloud API keys, network latency, and monthly costs.

---

3. Line-by-Line Code Breakdown

`app/pdf_loader.py`

```
from pypdf import PdfReader

def load_pdf(file_path):
    reader = PdfReader(file_path)  # Line 5: Initialize PDF reader object
    text = ""                      # Line 7: Accumulator string
    for page in reader.pages:       # Line 9: Loop over each page
        text += page.extract_text() or "" # Line 10: Extract page text safely
    return text
```

* **Line 5**: `PdfReader(file_path)` parses the PDF cross-reference table.

* **Line 10**: `extract_text() or ""` handles empty pages (like images) without returning `None` errors.

`app/text_splitter.py`

```
def split_text(text, chunk_size=100, overlap=20):
    words = text.split()            # Line 2: Convert string to list of words
    chunks = []
    start = 0
    while start < len(words):
        end = start + chunk_size
        chunk = " ".join(words[start:end]) # Line 11: Join slice into chunk string
        chunks.append(chunk)
        start += chunk_size - overlap       # Line 14: Advance start pointer by (chunk_size - ov
erlap)
    return chunks
```

* **Line 14**: `start += chunk_size - overlap` implements the sliding window! Step size is $100 - 20 = 80$ words forward per step.

`app/embeddings.py`

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("all-MiniLM-L6-v2") # Line 4: Load model into memory

def create_embeddings(chunks):
    embeddings = model.encode(chunks) # Line 8: Vectorize all chunks in parallel batch
    return embeddings
```

* **Line 8**: `model.encode(chunks)` returns a NumPy 2D array of shape `(N, 384)`.

`app/vector_store.py`

```
import chromadb
client = chromadb.PersistentClient(path="./chroma_db") # Line 4: Connect to disk storage
collection = client.get_or_create_collection(name="vidya_documents") # Line 6: Get/create collec
tion

def store_chunks(chunks, embeddings):
    ids = [f"chunk_{i}" for i in range(len(chunks))] # Line 12: Unique string IDs
    embeddings_list = embeddings.tolist() if hasattr(embeddings, "tolist") else embeddings # Lin
e 14: Convert NumPy array to list
    collection.add(ids=ids, documents=chunks, embeddings=embeddings_list) # Line 16: Index in Ch
romaDB
```

* **Line 4**: `PersistentClient` persists data to disk so index survives restarts.

* **Line 16**: `.add()` automatically constructs the spatial indexing tree.

---

4. Architectural Alternatives & Tradeoffs Summary

| Feature | Selected Choice | Alternative Option | Why Selected over Alternative? |

| :--- | :--- | :--- | :--- |

| **PDF Extraction** | `pypdf` | `pdfplumber` | Pure Python, fast, lightweight. |

| **Text Chunking** | Sliding Word Window | Character Cutoff | Preserves full words and semantic boundaries. |

| **Embeddings** | `all-MiniLM-L6-v2` | OpenAI `text-embedding-3-small` | Local CPU execution, 0 cost, 0 network latency. |

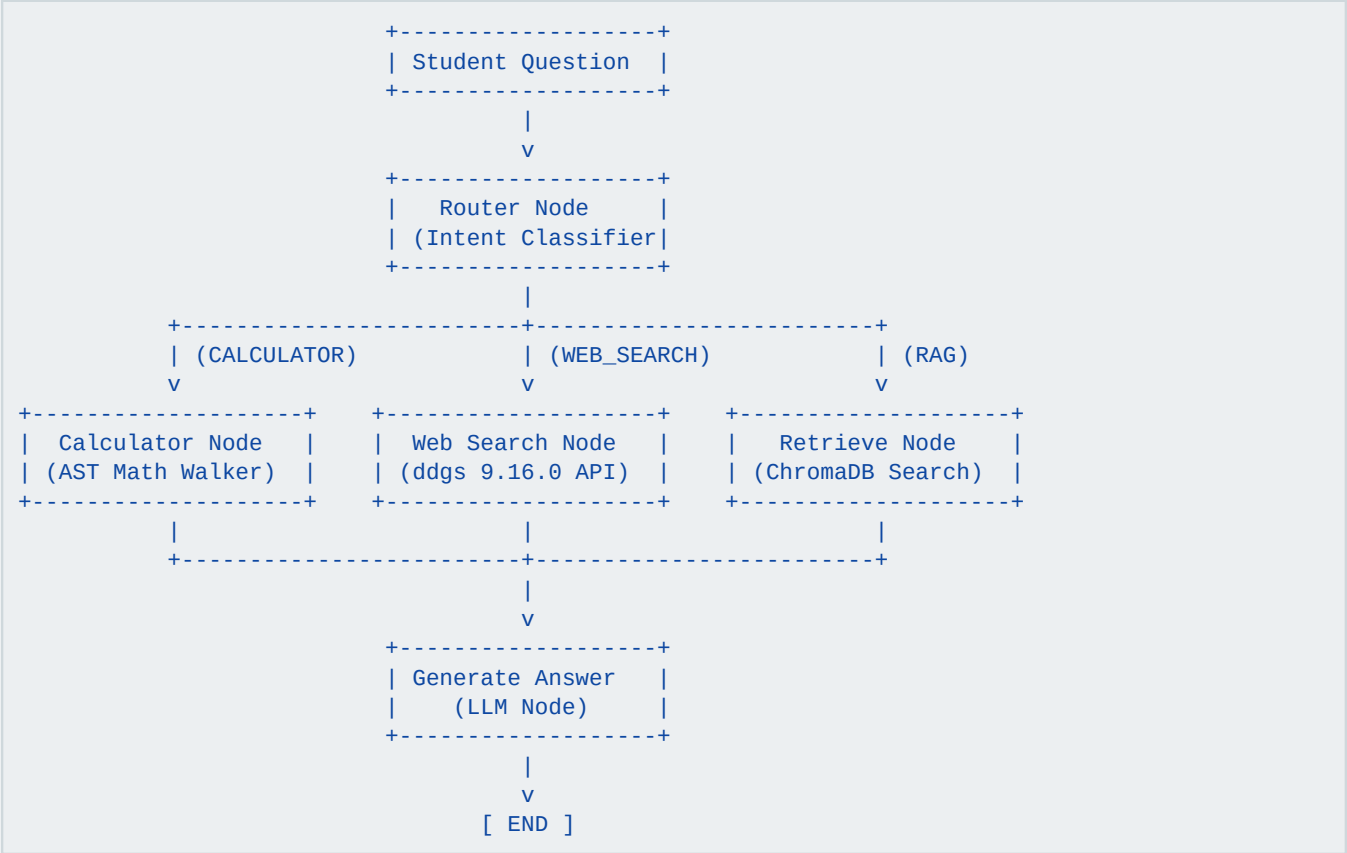
| **Vector Storage** | ChromaDB Persistent | Python In-Memory List | Disk persistence, HNSW fast indexed similarity search vs $O(N)$ scan. |

Chapter 2: Milestone 9 — LangGraph State Engine Architecture

1. High-Level Architecture Overview

In V1, pipeline execution was scripted sequentially (`load -> split -> embed -> store`). As VIDYA expanded to support multi-capability tutoring, sequential scripting became rigid and unmaintainable.

Milestone 9 introduced **LangGraph**, transforming VIDYA into a stateful, event-driven graph engine where steps are **Nodes**, data is held in a central **State**, and flow control is managed by **Edges and Routers**.



---

2. Core Concepts & Topics Used

A. Graph State (`VIDYAState` - `TypedDict`)

- **What it is**: A central schema dictionary that acts as the single source of truth passed to every node.

- **Why** `TypedDict`: Provides full IDE autocomplete and static type safety without runtime overhead.

- **How State Flow Works**: When a node runs, it receives the current `VIDYAState` dictionary and returns *only the keys it updated*. LangGraph automatically merges these updates back into the state.

B. Nodes (`load_pdf_node`, `router_node`, `calculator_node`, etc.)

- **What it is**: Pure Python wrapper functions that perform one specific task.

- **Design Principle**: Decoupling. Core logic remains in standalone functions (`load_pdf`, `calculator_tool`), while nodes simply map inputs from `state` and write results back to `state`.

C. Conditional Routing Edges (`select_route`)

- **What it is**: Dynamic decision functions that inspect `state["route_choice"]` and return the name of the next target node.

- **Why Router vs Sequential** `if/else`:

- **Sequential** `if/else`: Hardcodes paths inside single monolithic functions, making it hard to inspect, test, or re-route.

- **LangGraph Routers**: Decouple routing rules from execution logic. You can visualize, intercept, and modify graph flows programmatically.

---

3. Line-by-Line Code Breakdown

`app/graph_state.py`

```
from typing import TypedDict, Optional

class VIDYAState(TypedDict):
    pdf_path:       str              # Input PDF path
    pdf_text:       Optional[str]    # Extracted text
    chunks:         Optional[list]   # Chunks list
    embeddings:     Optional[list]   # Embeddings list
    query:          Optional[str]    # User query
    context_chunks: Optional[list]   # Retrieved RAG context
    answer:         Optional[str]    # Final answer
    route_choice:   Optional[str]    # Router decision: RAG, CALCULATOR, WEB_SEARCH, GENERAL
    tool_used:      Optional[str]    # Name of executed tool
    tool_result:    Optional[str]    # Output of executed tool
```

* `TypedDict`: Ensures every node knows exactly what fields exist in the graph notebook.

`app/nodes.py` (Router & Tool Nodes)

```
def router_node(state: VIDYAState) -> dict:
    query = state.get("query", "")
    route = classify_intent(query)  # Line 72: Call LLM / keyword intent classifier
    return {"route_choice": route}  # Line 74: Return state update dictionary

def calculator_node(state: VIDYAState) -> dict:
    query = state.get("query", "")
    result = calculator_tool(query) # Line 82: Run safe math evaluator
    return {"tool_used": "calculator", "tool_result": result}
```

`app/graph.py` (Graph Construction & Compilation)

```
from langgraph.graph import StateGraph, END

def build_qa_graph():
    builder = StateGraph(VIDYAState)
    builder.add_node("router",          router_node)
    builder.add_node("calculator",      calculator_node)
    builder.add_node("web_search",       web_search_node)
    builder.add_node("retrieve",        retrieve_node)
    builder.add_node("generate_answer", generate_answer_node)

    builder.set_entry_point("router") # Entry node

    builder.add_conditional_edges(
        "router",
        select_route, # Inspects state["route_choice"]
        {
            "calculator":      "calculator",
            "web_search":       "web_search",
            "retrieve":        "retrieve",
            "generate_answer": "generate_answer",
        },
    )

    builder.add_edge("calculator",      "generate_answer")
    builder.add_edge("web_search",       "generate_answer")
    builder.add_edge("retrieve",        "generate_answer")
    builder.add_edge("generate_answer", END)

    return builder.compile()
```

---

4. Architectural Alternatives & Tradeoffs Summary

| Feature | Selected Choice | Alternative Option | Why Selected over Alternative? |

| :--- | :--- | :--- | :--- |

| **Pipeline Control** | LangGraph StateGraph | Sequential `if-else` script | Modular state management, dynamic conditional routing, clean tool additions. |

| **State Management**| `TypedDict` | Custom Class / Pydantic | Lightweight, fast dictionary performance, native Python compatibility. |

| **Tool Execution** | Graph Nodes | Monolithic LLM Callback | Explicit control flow, predictable debugging, clear node tracing. |

Chapter 3: V2.1 — Deterministic Calculator Tool & Safe AST Evaluation

1. High-Level Architecture Overview

LLMs are probabilistic token predictors — they guess the next word based on probability patterns. When asked math questions like *"What is 17.5% of 85,000?"*, LLMs often hallucinate arithmetic answers.

V2.1 introduces a **Deterministic Calculator Tool** that routes math queries away from LLM guesswork into a **Safe Abstract Syntax Tree (AST) Evaluator**.

```
Student Query: "Calculate 17.5% of 85,000"
                ■
                ▼
[Router Node] ---> Intent detected: "CALCULATOR"
                ■
                ▼
[Calculator Tool]
   ■■■ 1. Regex Preprocessor: "17.5% of 85000" -> "(17.5 / 100) * 85000"
   ■■■ 2. AST Parser: ast.parse("0.175 * 85000", mode='eval')
   ■■■ 3. Safe AST Walker: Recursively evaluates ONLY safe Math Operators
                ■
                ▼
Result: "14,875" -> Passed to [Generate Answer Node] -> LLM presents formatted answer
```

---

2. Core Concepts & Topics Used

A. Abstract Syntax Trees (AST)

- **What it is**: A tree representation of the syntactic structure of source code.

- **Example**: `5 + 3` becomes:

`

BinOp (Add)

/ Constant(5) Constant(3)

`

B. Security & Safe Evaluation vs Python `eval()`

- **Dangers of** `eval()`:

Using Python's built-in `eval("17.5 * 850")` is extremely dangerous. An attacker passing `"__import__('os').system('rm -rf /')"` would execute arbitrary code on your server!

- **Safe AST Walker Solution**:

Our custom walker inspects every node in the AST tree before executing it. If a node is an import, function call, loop, or variable reference, it raises `ValueError("Unsupported AST node")`. It ONLY permits numeric constants and safe math operators (`+`, `-`, `*`, `/`, `%`, `**`).

---

3. Line-by-Line Code Breakdown

`app/tools.py` (`_safe_eval_ast` & `safe_evaluate`)

```
import ast
import operator
import re

SAFE_OPERATORS = {
    ast.Add: operator.add,
    ast.Sub: operator.sub,
    ast.Mult: operator.mul,
    ast.Div: operator.truediv,
```

```
    ast.FloorDiv: operator.floordiv,
    ast.Mod: operator.mod,
    ast.Pow: operator.pow,
    ast.USub: operator.neg,
    ast.UAdd: operator.pos,
}

def _safe_eval_ast(node):
    if isinstance(node, ast.Constant):      # Line 32: Numbers (int, float)
        if isinstance(node.value, (int, float)):
            return node.value
        raise ValueError(f"Unsupported constant: {type(node.value)}")

    elif isinstance(node, ast.BinOp):       # Line 37: Binary operations (e.g. 5 + 3)
        op_type = type(node.op)
        if op_type in SAFE_OPERATORS:
            left = _safe_eval_ast(node.left)
            right = _safe_eval_ast(node.right)
            return SAFE_OPERATORS[op_type](left, right) # Line 42: Apply operator
        raise ValueError(f"Unsupported operator: {op_type}")

    raise ValueError(f"Unsupported AST node: {type(node)}") # Line 51: Security block!
```

* **Line 42**: `SAFE_OPERATORS[op_type](left, right)` maps the AST operator (`ast.Add`) directly to Python's math function (`operator.add(left, right)`).

* **Line 51**: Anything non-math (e.g. `__import__`, `open()`, `eval()`) hits Line 51 and is immediately blocked.

Preprocessing & Formatting (`safe_evaluate` & `calculator_tool`)

```
def safe_evaluate(expression_str: str) -> float:
    # 1. Convert percentages: "17.5% of 85000" -> "(17.5 / 100) * 85000"
    expr = re.sub(r'(\d+(?:\.\d+)?)\s*%\s*of\s*(\d+(?:\.\d+)?)', r'(■ / 100) * ■', expression_s
tr, flags=re.IGNORECASE)
    expr = re.sub(r'(\d+(?:\.\d+)?)\s*%', r'(■ / 100)', expr)

    # 2. Convert word symbols
    expr = expr.replace("×", "*").replace("÷", "/").replace("x", "*")

    # 3. Strip invalid non-math characters
    cleaned = re.sub(r'[^0-9\+\-\*\/\%\.\(\)\s]', '', expr).strip()

    parsed = ast.parse(cleaned, mode='eval')
    return _safe_eval_ast(parsed.body)
```

---

4. Architectural Alternatives & Tradeoffs Summary

| Feature | Selected Choice | Alternative Option | Why Selected over Alternative? |

| :--- | :--- | :--- | :--- |

| **Math Execution** | Safe AST Walker | Python `eval()` | `eval()` has catastrophic Remote Code Execution (RCE) vulnerabilities. AST walker is 100% safe. |

| **Math Handling** | Deterministic CPU Tool | Direct LLM Guesswork | LLMs hallucinate math operations. CPU AST walker guarantees 100% precision. |

Chapter 4: V2.2 — Web Search Capability & `ddgs` 9.16.0 Integration

1. High-Level Architecture Overview

RAG is limited to the uploaded PDF. When a student asks about current events or external developments (*"What is the latest Python version?"*), RAG cannot answer.

V2.2 adds **Web Search Capability** using DuckDuckGo Search via the modern `ddgs` 9.16.0 API.

```
Student Query: "Search the web for latest Python version"
                ■
                ▼
[Router Node] ---> Intent detected: "WEB_SEARCH"
                ■
                ▼
[Web Search Node]
   ■■■ Clean query: "latest Python version"
   ■■■ Call `ddgs.text(clean_query, max_results=4)`
   ■■■ Format structured results: Title, URL, Snippet
                ■
                ▼
Structured Web Context passed to [Generate Answer Node] ---> LLM answers with real-time web facts!
```

---

2. Core Concepts & Package Namespace Resolution (`ddgs` vs `duckduckgo_search`)

Why `from ddgs import DDGS` is the Correct Choice

- In older releases (v5-v7), DuckDuckGo Search was distributed under the PyPI package `duckduckgo_search`.

- In modern v9.x releases (such as installed version `9.16.0`), the official PyPI package was renamed to `ddgs`.

- Importing `from duckduckgo_search import DDGS` in v9.x outputs internal `RuntimeWarning: This package (duckduckgo_search) has been renamed to ddgs!`.

- Importing directly via `from ddgs import DDGS` uses the clean, modern namespace without deprecation warnings.

---

3. Line-by-Line Code Breakdown

`app/tools.py` (`web_search_tool`)

```
from ddgs import DDGS  # Line 11: Direct clean import from ddgs package

def web_search_tool(query: str, max_results: int = 4) -> str:
    clean_query = query
    for prefix in ["search the web for", "search online for", "search the web", "search online",
 "search for"]:
        if clean_query.lower().startswith(prefix):
            clean_query = clean_query[len(prefix):].strip(" :")

    try:
        with DDGS() as ddgs:
```

```
            results = list(ddgs.text(clean_query, max_results=max_results))

        if not results:
            return f"No web search results found for query '{clean_query}'."

        formatted = []
        for i, item in enumerate(results, 1):
            title = item.get("title", "No Title")
            url = item.get("href", item.get("link", "#"))
            snippet = item.get("body", item.get("snippet", "No Snippet"))
            formatted.append(f"[{i}] {title}\nURL: {url}\nSnippet: {snippet}")

        return "\n\n".join(formatted)
    except Exception as e:
        return f"[Web Search Note]: Search failed ({str(e)}). Unable to verify live web data."
```

* **Line 102**: `with DDGS() as ddgs:` opens a clean HTTP session context manager.

* **Line 103**: `ddgs.text(clean_query, max_results=4)` fetches live search results.

* **Line 118-121**: Converts raw JSON response dictionaries into human-readable formatted context blocks for the LLM.

---

4. Architectural Alternatives & Tradeoffs Summary

| Feature | Selected Choice | Alternative Option | Why Selected over Alternative? |

| :--- | :--- | :--- | :--- |

| **Search Engine API** | `ddgs` 9.16.0 | Google Custom Search API / Tavily | Free, open-source, no API keys required, zero billing cost. |

| **Import Namespace** | `from ddgs import DDGS` | `from duckduckgo_search import DDGS` | Clean, warning-free, uses standard v9.16.0 package namespace. |